

Modules ABI Requirement

Document #: D3092R0
Date: 2024-01-19
Project: Programming Language C++
Audience: SG15, ABI Review Group
Reply-to: Chuanqi Xu
<chuanqi.xcq@alibaba-inc.com>

Contents

- [1 Abstract](#)
- [2 Motivation](#)
- [3 Simple introduction for modules](#)
- [4 New requirement to ABI](#)
 - [4.1 Module Initializers](#)
 - [4.2 Module Linkage](#)
- [5 ABI related change in the language side](#)
 - [5.1 TU-locals](#)
 - [5.2 in-class member functions is not implicitly inline in modules purview](#)
- [6 Std modules](#)
- [7 Wishes](#)
 - [7.1 ABI boundaries](#)

§ 1 Abstract

C++20 introduces a new language construct Modules. Modules has non-trivial implications for ABIs. Although we tried to not break previous ABI specification and we made it, it is still helpful to describe the requirement to ABI for modules precisely so that the ABI specication can understand what is allowed to change and what is not allowed.

§ 2 Motivation

The motivation of the paper is a discussion about how to define virtual tables in modules: <https://github.com/itanium-cxx-abi/cxx-abi/issues/170>.

Prior to modules, the virtual table is emitted in the same object containing the definition of its key function, i.e. the first non-pure virtual function that is not inline at the point of class definition.

The rule can work well even after modules come in. However, the ABI can get rid of the concept of key functions within modules. It can simplify the mental model and the implementations.

This is a good example why this document is needed. While we introduce a new language construct and the old ABI rules could work well, the ABI rules can get improved for modules after they understand new construct well.

§3 Simple introduction for modules

Modules allow us to get informations from other (importable) module units.

e.g.,

```
// a.cppm
export module a;
export int a() { return 43; }

// b.cpp
import a;
int b() { return a(); }
```

(In b.cpp, we can call function a without declaring it earlier in the current TU.)

Module units are new translation unit kinds, including:

- Primary module interface unit. A translation unit containing a module declaration `export module <module-name>;`. <module-name> should be in the form of `[a-zA-Z_][a-zA-Z_0-9\.]*`. In a valid program, there shouldn't be multiple primary module interface units with the same <module-name>.
- Module implementation unit. A translation unit containing a module declaration `module <module-name>;`. There can be multiple module implementation units with the same <module-name>. The module implementation units will import the corresponding primary module interface unit implicitly.

- Module interface partition unit. A translation unit containing a module declaration `export module <module-name>:<partition-name>;`. `<partition-name>` has the same form with `<module-name>`. In a valid program, there shouldn't be multiple module partition units with the same `<module-name>:<partition-name>` pair. All the module interface partition unit should directly or indirectly be exported by the corresponding primary module interface unit.
- Module internal module partition unit. A translation unit containing a module declaration `module <module-name>:<partition-name>;`

Every module unit should have exactly one module declaration.

Each module unit consists of the following form:

```
[<global-module-fragment>]
<module-declaration>
...
[module :private;] // optional
```

A global module fragment is an optional section in the following form:

```
module;
...
```

The global module is the collection of all global-module-fragments and all translation units that are not module units. Declarations appearing in such a context are said to be in the purview of the global module.

The section from `<module-declaration>` to the end of the module unit is called module unit purview. The purview of a

named module `M` is the set of module unit purviews of `M`'s module units.

Every declaration are either attached to the global module or a named module. The rules are described here [module.unit/p7](#):

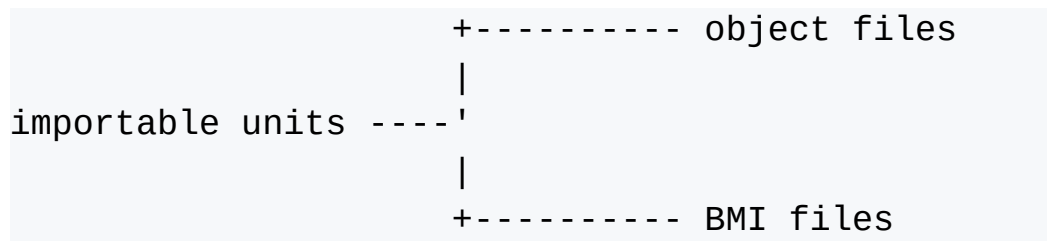
- If the declaration is a non-dependent friend declaration that nominates a function with a declarator-id that is a qualified-id or template-id or that nominates a class other than with an elaborated-type-specifier with neither a nested-name-specifier nor a simple-template-id, it is attached to the module to which the friend is attached.
- Otherwise, the declaration is attached to the global module if
 - is a namespace-definition with external linkage or
 - appears within a linkage-specification
- Otherwise, the declaration is attached to the module in whose purview it appears.

The section under `module :private;` is called private module fragment. The private module fragment can only appear in a primary module interface unit. And a primary module interface unit containing a private module fragment should be the only module unit of the corresponding module. The entities in private module fragment won't affect other translation units. We can think the entities in private module fragment as if they are in an seperate module implementation unit.

The module purviews of module units with same `<module-name>` consist a module with `<module-name>`.

The primary module interface unit, module interface partition unit and module internal partition unit are called importable module unit.

The importable module unit should be compiled into object files and BMI (Built Module Interface) files. The format of BMI files is implementation defined.



§ 4 New requirement to ABI

This section describes the requirement of modules to ABI specification.

There is already an implementation in clang and GCC and there is pull request to add this to Itanium C++ ABI: <https://github.com/itanium-cxx-abi/cxx-abi/pull/144>

§ 4.1 Module Initializers

All the importable module units are required to emit an initializer function. The initializer function should contain calls to importing modules first and all the dynamic-initializers in the current module unit then.

Translation units explicitly or implicitly importing named modules must call the initializer functions of the imported named modules within the sequence of the dynamic-initializers in the TU. Initializations of entities at namespace scope are appearance-ordered. This (recursively) extends

into imported modules at the point of appearance of the import declaration.

It is allowed to omit calls to importing modules if it is known empty.

It is allowed to omit calls to importing modules for which is known to be called.

§ 4.2 Module Linkage

The language specification introduces a new linkage module linkage.

All non `TU-local` (See below) entities attached to the purview of named modules, which don't get external linkage by other means, has module linkage. When a name has module linkage, the entity it denotes can be referred to by names from other scopes of the same module unit or from scopes of other module units of that same module.

(Note: 'Inline' doesn't change attachment and therefore doesn't affect linkage in this respect.)

In clang and GCC, we implement module linkage by introducing new mangle names. See <https://github.com/itanium-cxx-abi/cxx-abi/pull/144> for details.

§ 5 ABI related change in the language side

This section describes the change in the language side but not requiring the ABI spec to change.

§ 5.1 TU-locals

Module units are translation units that can be imported. Then we should avoid the internal linkage get imported into other translation units.

To address the idea, we bring the concept of TU-locals and exposure to the language. The formal definitions to TU-locals and exposure are [basic.link/p14](#), [basic.link/p15](#), [basic.link/p16](#), [basic.link/p17](#) and [basic.link/p18](#).

We can think TU-locals as the entities which should be only usable in the module unit and the exposure are declarations which leak the the TU-locals.

The exposures are not allowed to appear in any importable module unit (ignoring private module fragment, if any).

An interesting point here is, we don't count the body of non-inline functions (and function templates) for deciding exposure.

```
export module a;  
static int local() { ... }  
export int external() { return local(); }
```

Here the function external is not an exposure even if its body contains a call to a TU-local declaration.

This implies that the implementation shouldn't import the bodies of non-inline functions into the consumers, even if in optimizations. Otherwise, it is problematic if the static entities get visible to other TUs.

Another interesting point is the bodies of function templates don't count too.


```
export module a;
static int local() { ... }
export template<int>
int external() { return local(); }
```

The above program is valid too. We don't think the template `external` as an exposure. This is useful with template specializations and explicit template instantiations.

```
// a.cppm
export module a;
static int local() { ... }
export template<int>
int external() { return local(); }
export template int external<0>();

// b.cpp
import a;
int other() {
    return external<0>() // Valid.
        + external<1>(); // Invalid.
}
```

The rationale behind the rule is, with explicit template instantiations, the function bodies of `external<0>()` is invisible to `b.cpp`. Then it is fine. But for `external<1>()`, its function bodies is visible to `b.cpp` due to implicit instantiations in `b.cpp`. So it is invalid.

§ 5.2 in-class member functions is not implicitly inline in modules purview

According to [dcl.inline/p4](#):

In the global module, a function defined within a class definition is implicitly inline ([class.mfct], [class.friend]).

In other word, the in-class function definitions in the module purview is not implicitly inline.

§ 6 Std modules

The C++ standard library provides the `std` module and `std.compat` module [std.module](#). This section describes the ABI requirement for these two modules.

The `std` and `std.compat` module are reserved module that user shouldn't define. So it leaves the space for compilers to do special tricks for the `std` and `std.compat` modules. But no implementation does that by the time of writing.

It is unspecified to which module a declaration in the standard library is attached. But implementations are required to ensure that mixing `#include` and `import` does not result in conflicting attachments. This implies that the declarations in the `std` and `std.compat` module should have same linkages and the same mangled names as in the header.

§ 7 Wishes

This section describes the ABI-related wishes to modules which is not reflected in the wording of specification.

§ 7.1 ABI boundaries

We wish the definitions of non-inline functions and non-inline variables in modules won't affect ABI boundaries.

That said, after we change the definitions of non-inline functions in an importable module unit, it is allowed to skip

the recompilations of all the consumers of that module unit. While no compiler and build system implemented this yet, we think this is a promising feature to improve the compilation speed of modules.

This implies that the bodies of non-inline functions can't get inlined into functions in other units without LTO, which is possible by importing the bodies as `available_externally` in LLVM.